



Training Python-Django - 3

Final

Summary: Today, we're gonna learn how to use AJAX and Websockets with Django.

Version: 1.2

Contents

I	General rules	2
II	Today's specific rules	3
III	Exercise 00	4
IV	Exercise 01	6
V	Exercise 02	8
VI	Exercise 03	9
VII	Exercise 04	10
VIII	Submission and peer-evaluation	11

Chapter I

General rules

- Your project must be realized in a virtual machine.
- Your virtual machine must have all the necessary software to complete your project. These softwares must be configured and installed.
- You can choose the operating system to use for your virtual machine.
- You must be able to use your virtual machine from a cluster computer.
- You must use a shared folder between your virtual machine and your host machine.
- During your evaluations you will use this folder to share with your repository.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc) apart from undefined behaviors. If this happens, your project will be considered non functional and will receive a 0 during the evaluation.
- We encourage you to create test programs for your project even though this work **won't have to be submitted and won't be graded**. It will give you a chance to easily test your work and your peers' work. You will find those tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to your assigned git repository. Only the work in the git repository will be graded. If Deepthought is assigned to grade your work, it will be done after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.



For obvious security reasons, any credentials, API keys, env variables etc... must be saved locally in a `.env` file and ignored by git. Publicly stored credentials will lead you directly to a failure of the project.

Chapter II

Today's specific rules

- The only JavaScript library you can use is `jQuery`.
- Your turn-in will take the form of a unique `Django` project. It won't be divided into exercises, as usual. Each of them adds a functionality to the project. This functionality and its implementation will be graded this time.
- You must leave the default administration application.
- Along with your project, you must turn in a `requirements.txt` file (with the `'pip freeze'` command) listing the libraries your project will need to be run.

Chapter III

Exercise 00

	Exercise00
Exercise 00: AJAX my formula!	
Directory: <i>ex00/</i>	
Files to Submit:	
Authorized: None	

Create a new project named `d09`. In this project, create an application named `account`.

The goal of this exercise is to design a login/logout system communicating only through AJAX.

In this application, you will implement the `127.0.0.1:8000/account` URL that must lead to a page that can have two different behaviors depending on the context:

- The user is not connected: the page must display a standard login form (login, password). The thing is, the communication with the server must only use AJAX and must be a `POST` request to log in.

If the form is not valid, the error(s) must appear on the page.

If the form is valid, it must disappear and adopt a new behavior.

This, of course, without the page being refreshed.

- The user is already connected: the page must display the following text: "Logged as `<user>`", `<user>` being replaced by the name used by the user to log in, as well as a `Logout` button allowing disconnection.

This button must communicate with the server via AJAX and the `POST` method.

Once logged out, the text and button must no longer appear on the page and the latter must adopt the other behavior.

The page must never be refreshed.

If the page is manually refreshed, it must return to the behavior it had before refreshing (that does not include the error display).

You can use Bootstrap.



`AuthenticationForm`, for free!

Chapter IV

Exercise 01

	Exercise01
Exercise 01: Basic chat	
Directory: <i>ex01/</i>	
Files to Submit:	
Authorized: None	

Create an application named '**chat**'.

In this application, you must create a page displaying 3 links. Each of them must lead to a different '**chatroom**'.

The names of these rooms must be in the database. You must create a suitable model.

Each of these links must lead to another page containing a standard functional chat. Each chat must have the following specifications:

- It must use '**jquery**' as its sole frontend library as well as **WebSockets** to communicate with the server (no AJAX).
- It's only available to logged-in users.
- The name of the chat must appear somewhere.
- Several users must be able to connect (*just in case...*).
- A user can post a message (*you had guessed, right?*).
- A message sent by a user must be visible to all the users who have joined the chatroom (*everyone knows what a chatroom is, right? Haven't you read that preamble?*).
- Messages must appear at the bottom and be displayed in ascending order (*that one's for you, by the heater, right.*), along with the name of the user who posted them.

- Messages must not disappear. A message must not replace a previous one. The order of the messages must not change.
- When a user joins the chatroom, the message '`<username> has joined the chat`' must appear for all users to see, including the one who just joined. `<username>` is replaced by said user's name, of course.

Chapter V

Exercise 02

	Exercise02
Exercise 02: History	
Directory: <i>ex02/</i>	
Files to Submit:	
Authorized: None	

In this exercise, you will improve your chat by adding a message history.

When a new user joins the chatroom, they must see the last three messages that have been posted in this chatroom, from oldest to newest.

Once again, you can only use **JQuery** as your frontend library and **WebSockets** to communicate with the server.

Chapter VI

Exercise 03

	Exercise03
Exercise 03: User list	
Directory: <i>ex03/</i>	
Files to Submit:	
Authorized: None	

In this exercise, you will improve your chat again by adding a connected user list, this time. AND it will update by itself.

When the user joins the chatroom, they must be able to access the list of connected users (and they must appear in the list).

This user list must be clearly set aside from the messages list (other `<div>` or other HTML container).

When a user joins a chatroom, their name must appear in the list, along with other connected users.

When a user leaves the chatroom, their name must disappear from the list of connected users and the message '`<username>` has left the chat' must appear after the posted messages. (`<username>` will be the name of the user who just left.)

Once again, you can only use **JQuery** as a frontend library and **WebSockets** to communicate with the server.



Before anything, try to build a functional logic. We're not looking for optimization... yet.

Chapter VII

Exercise 04

	Exercise04
Exercise 04: Scroll	
Directory: <i>ex04/</i>	
Files to Submit:	
Authorized: None	

Make your chat presentable by setting your message list in a fixed-size container. If the number of messages exceeds the container, they must disappear at the top and a scroll bar must appear on the side.

Besides, the scroll bar must always appear with the cursor at the bottom so the last messages always show first.

Chapter VIII

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double-check the names of your folders and files to ensure they are correct.



The evaluation process will happen on the computer of the evaluated group.